

Cụm 9 - Seminar Advanced Topics

Mục lục

9.1 Tổng quan Seminar - Advanced Topics	3
Vi sao cần cụm seminar?	3
Bốn chủ đề	4
Cách đọc cụm này	4
Nếu bạn muốn học nhanh	4
Nếu bạn là Data Scientist	4
Nếu bạn là backend/platform engineer	4
Framework chung cho bốn seminar	5
Mapping từ nền tảng sang seminar	5
Điều quan trọng nhất	5
Bài tiếp	5
9.2 Software Architecture for IoT	6
1. Domain context	6
2. Functional requirements	6
Device onboarding	6
Telemetry ingestion	6
Command and control	6
Firmware update	6
Operations	7
3. Quality Attributes	7
4. Architecture style mix	7
5. High-level architecture	9
6. Edge-cloud split	9
7. Telemetry pipeline	9
8. Command delivery flow	9
9. Device identity and security	10
Design principles	10
Security boundaries	10
10. Data strategy	10
11. Failure modes	11
12. ADR examples	11
ADR 1: Use MQTT broker and event bus for telemetry	11
ADR 2: Put safety-critical rules at edge	11
ADR 3: Use tiered storage for telemetry	12
13. What to document	12
14. Self-check	12

15. Tóm tắt	12
9.3 Software Architecture for Web3	13
1. Domain context	13
2. Functional requirements	13
Marketplace	13
Verification	13
Wallet and identity	13
Indexing and search	13
Compliance	14
3. On-chain vs off-chain decision	14
4. Quality Attributes	14
5. Architecture style mix	15
6. High-level architecture	15
7. Critical flow: buying a credit	16
8. Indexer pipeline	16
9. Smart contract as architectural boundary	17
Contract invariants	17
10. Upgrade strategy	17
11. Security model	17
12. Privacy and data placement	18
13. Failure modes	18
14. ADR examples	19
ADR 1: Store token ownership on-chain, documents off-chain	19
ADR 2: Build an indexer instead of querying chain directly from UI	19
ADR 3: Use multisig and timelock for contract upgrades	19
15. What to document	19
16. Self-check	20
17. Tóm tắt	20
9.4 Software Architecture for MLOps	21
1. Domain context	21
2. Functional requirements	21
Data and feature	21
Experiment and training	21
Deployment and serving	21
Monitoring and feedback	22
Governance	22
3. Quality Attributes	22
4. Architecture style mix	22
5. High-level architecture	23
6. Core platform boundaries	23
7. Training-to-serving lifecycle	23
8. Deployment modes	23
9. Model registry lifecycle	25
10. Monitoring design	25
11. Governance gates	25
12. Failure modes	26
13. ADR examples	26

ADR 1: Use model registry as lifecycle authority	26
ADR 2: Support multiple serving modes	26
ADR 3: Enforce feature version compatibility at serving load time	28
14. Documentation checklist	28
15. Self-check	28
16. Tóm tắt	28
9.5 Software Architecture for Digital Twin	29
1. Domain context	29
2. Physical twin vs digital twin	29
3. Functional requirements	29
Twin modeling	29
Real-time sync	29
Analytics and simulation	30
Visualization	30
Closed-loop action	30
4. Quality Attributes	30
5. Architecture style mix	30
6. High-level architecture	31
7. Twin state update flow	31
8. Digital thread	31
9. Simulation architecture	32
10. Open-loop vs closed-loop control	33
11. Safety and command policy	33
12. Data modeling	33
13. Consistency and freshness	34
14. Failure modes	34
15. ADR examples	34
ADR 1: Use event-driven state synchronization	34
ADR 2: Use graph model for asset topology	35
ADR 3: Require human approval for high-risk commands	35
16. What to document	35
17. Self-check	35
18. Tóm tắt	36

9.1 Tổng quan Seminar - Advanced Topics

Tóm tắt một dòng: Cụm 9 không dạy thêm một style mới. Cụm này dạy cách dùng toàn bộ nền tảng từ Cụm 1-8 để phân tích bốn domain khó: IoT, Web3, MLOps và Digital Twin.

Vì sao cần cụm seminar?

Sau khi học xong 8 cụm nền tảng, bạn đã có vocabulary để nói về:

- Architectural decisions.
- Quality Attributes.
- Trade-off.
- Component boundary.
- Architecture styles.
- Documentation views.
- ADR.

Nhưng khi bước vào hệ thống hiện đại, câu hỏi hiếm khi xuất hiện dưới dạng sạch như “nên dùng layered hay microservices?”. Câu hỏi thực tế thường là:

- Hệ IoT có hàng trăm nghìn thiết bị offline liên tục thì kiến trúc ra sao?
- Hệ Web3 có smart contract immutable thì rollback như thế nào?
- Nền tảng MLOps có training, serving, monitoring, governance thì boundary đặt ở đâu?
- Digital Twin cần đồng bộ physical world và virtual model thì latency, consistency, simulation xử lý thế nào?

Bốn seminar này giúp bạn luyện kỹ năng **chuyển từ domain constraint sang architecture decision**.

Bốn chủ đề

Bài	Chủ đề	Trọng tâm kiến trúc
9.2	Software Architecture for IoT	Edge-cloud split, device fleet, unreliable network, telemetry pipeline
9.3	Software Architecture for Web3	Smart contracts, off-chain services, indexer, wallet, finality, governance
9.4	Software Architecture for MLOps	ML platform, pipeline orchestration, feature store, model registry, serving, monitoring
9.5	Software Architecture for Digital Twin	Real-time mirror, simulation, event streaming, time-series data, command loop

Cách đọc cụm này

Nếu bạn muốn học nhanh

Đọc theo thứ tự:

1. [9.2 IoT](#), vì nó nối trực tiếp với event-driven, pipeline và edge deployment.
2. [9.4 MLOps](#), nếu bạn đến từ Data Science hoặc data platform.
3. [9.5 Digital Twin](#), vì nó kết hợp IoT, streaming, simulation và visualization.
4. [9.3 Web3](#), vì nó có constraint đặc biệt nhất: immutability và trust boundary.

Nếu bạn là Data Scientist

Ưu tiên 9.4 trước, sau đó đọc 9.5. MLOps cho bạn thấy vòng đời model từ training tới serving và monitoring. Digital Twin cho bạn thấy ML model được đặt trong một hệ cyber-physical system lớn hơn, nơi prediction có thể tác động ngược lại physical world.

Nếu bạn là backend/platform engineer

Ưu tiên 9.2 và 9.3. IoT giúp bạn luyện edge-cloud partitioning, backpressure, device identity và telemetry ingestion. Web3 giúp bạn hiểu cách thiết kế hệ thống khi một phần business logic nằm trong smart contract không dễ thay đổi.

Framework chung cho bốn seminar

Mỗi bài đi qua cùng một khung:

1. Domain context.
2. Functional requirements.
3. Quality Attributes.
4. Style mix.
5. High-level architecture.
6. Critical runtime flows.
7. Data and state strategy.
8. Security and governance.
9. Failure modes.
10. ADR examples.
11. Self-check.

Khung này cố tình giống Cụm 8. Điểm khác là domain phức tạp hơn và trade-off sắc hơn.

Mapping từ nền tảng sang seminar

Nền tảng	Dùng trong seminar
Quality Attributes	Mỗi domain có QA driver khác nhau
Pipeline Architecture	Telemetry, ETL, training, simulation data flow
Event-Driven Architecture	IoT events, Web3 indexer, model monitoring, twin sync
Service-based Architecture	Platform core services và domain services
Microservices	Chỉ dùng khi domain/team/scale justify
Microkernel	Plugin analytics, model adapters, protocol adapters
C&C View	Runtime flow của event, command, prediction, transaction
Allocation View	Edge, cloud, blockchain, GPU cluster, simulator nodes
ADR	Ghi lại quyết định không hiển nhiên

Điều quan trọng nhất

Không có “kiến trúc IoT”, “kiến trúc Web3”, “kiến trúc MLOps” hay “kiến trúc Digital Twin” duy nhất. Mỗi domain chỉ làm nổi bật một tập constraint đặc biệt.

Kiến trúc tốt vẫn bắt đầu từ cùng một câu hỏi:

Quality Attributes nào thật sự drive hệ này, và ta chấp nhận hy sinh gì để đạt chúng?

Bài tiếp

Bắt đầu với [9.2 Software Architecture for IoT](#).

9.2 Software Architecture for IoT

Tóm tắt một dòng: IoT architecture là bài toán phân chia trách nhiệm giữa device, edge và cloud trong điều kiện network không ổn định, device resource hạn chế, security khó, telemetry rất lớn và command phải an toàn.

1. Domain context

Một công ty năng lượng muốn xây nền tảng quản lý **smart meters** và **industrial sensors** cho nhiều khách hàng doanh nghiệp. Hệ thống cần thu thập telemetry, phát hiện bất thường, gửi command cấu hình, cập nhật firmware và cung cấp dashboard vận hành.

Scale ban đầu:

- 200k devices trong năm đầu.
- Tăng lên 2M devices trong 3 năm.
- Mỗi device gửi telemetry mỗi 10 giây.
- Một số site có edge gateway.
- Network không ổn định, đặc biệt ở nhà máy hoặc khu vực xa.
- Device có vòng đời 5-10 năm, khó thay thế nhanh.

Điểm khác biệt của IoT so với web app thông thường: client không phải browser đáng tin cậy. Client là device ngoài thực địa, có thể offline, bị capture, chạy firmware cũ, mất điện, mất network hoặc gửi data sai.

2. Functional requirements

Device onboarding

- Đăng ký device mới.
- Gán device vào tenant/customer/site.
- Cấp certificate hoặc credential.
- Kiểm tra firmware version và hardware model.

Telemetry ingestion

- Nhận metric định kỳ: voltage, temperature, power usage, vibration.
- Validate schema và timestamp.
- Store raw telemetry cho audit.
- Aggregate data cho dashboard.
- Detect anomaly near-realtime.

Command and control

- Gửi command đổi cấu hình.
- Gửi command restart hoặc calibrate sensor.
- Theo dõi trạng thái command: pending, delivered, acked, failed, expired.
- Không gửi command nguy hiểm nếu device không đúng trạng thái.

Firmware update

- Rollout firmware theo batch.
- Canary theo site hoặc hardware model.

- Rollback nếu error rate tăng.
- Verify firmware signature.

Operations

- Dashboard health theo device, site, tenant.
- Alert khi device offline, telemetry stale, anomaly tăng.
- Audit trail cho command và firmware update.

3. Quality Attributes

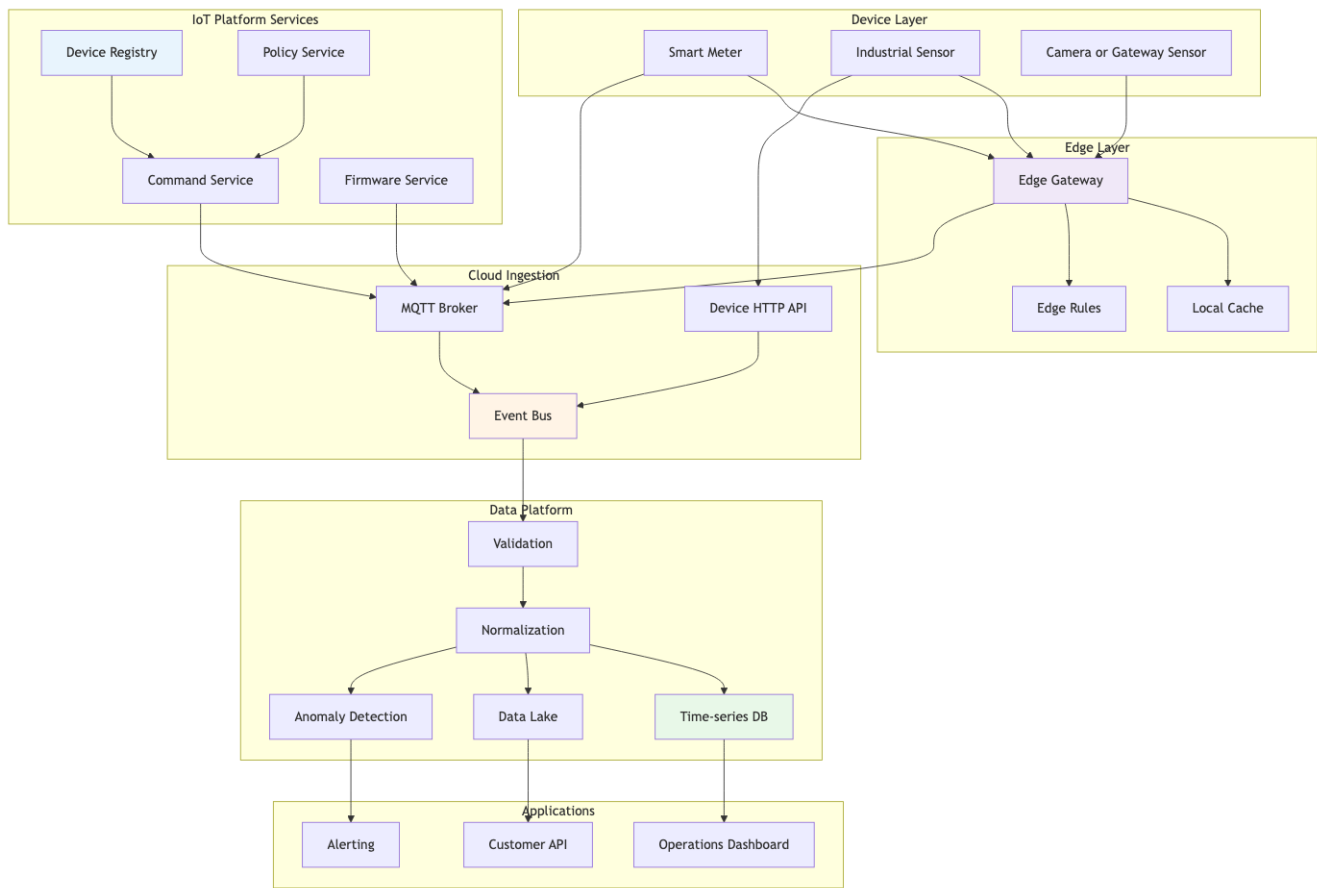
Priority	QA	Target
Must	Reliability	Không mất telemetry quan trọng khi network chậm chờn
Must	Scalability	2M devices, 200k events/s peak
Must	Security	Mutual auth, credential rotation, signed firmware
Must	Observability	Biết device nào stale, gateway nào lag, topic nào backlog
Must	Safety	Command nguy hiểm phải có guardrail và audit
Should	Latency	Telemetry dashboard p95 < 5s, critical alert < 30s
Should	Cost	Storage tiering để raw telemetry không phá budget
Should	Evolvability	Thêm protocol/device model không rewrite core

4. Architecture style mix

IoT platform hiếm khi dùng một style duy nhất.

Concern	Style phù hợp	Lý do
Telemetry ingestion	Event-Driven	Device gửi event bất đồng bộ, consumer scale độc lập
Data processing	Pipeline	Validate, normalize, enrich, aggregate, detect
Device management	Service-based	Device Registry, Command Service, Firmware Service
Protocol adapters	Microkernel	Thêm MQTT, CoAP, HTTP, LoRaWAN adapter như plugin
Dashboard/Admin	Layered hoặc service-based	CRUD, workflow, RBAC
Edge gateway	Mini pipeline + local cache	Chạy gần device, chịu offline

Decision tổng: **Event-Driven core + Pipeline telemetry + Service-based control plane + Edge gateway.**



Hình 1: Sơ đồ 60

5. High-level architecture

6. Edge-cloud split

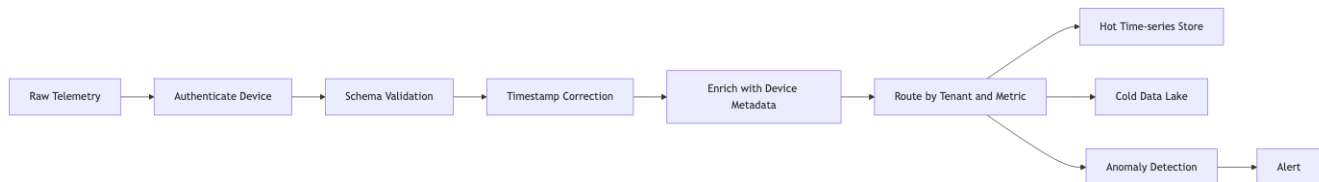
Một quyết định trung tâm trong IoT là đặt logic ở đâu.

Logic	Device	Edge	Cloud
Sensor reading	Có	Không	Không
Basic validation	Có	Có	Có
Local safety rule	Có nếu critical	Có	Không nên phụ thuộc
Aggregation	Ít	Có	Có
ML anomaly detection	Nhỏ nếu cần	Có thể	Có
Long-term storage	Không	Cache tạm	Có
Fleet analytics	Không	Không	Có
Firmware orchestration	Không	Relay	Có

Heuristic:

- Nếu quyết định cần phản ứng khi mất mạng, đưa xuống device hoặc edge.
- Nếu quyết định cần global view, để cloud.
- Nếu data volume quá lớn, aggregate ở edge trước khi gửi cloud.
- Nếu logic thay đổi thường xuyên, tránh nhúng sâu vào firmware trừ khi bắt buộc.

7. Telemetry pipeline



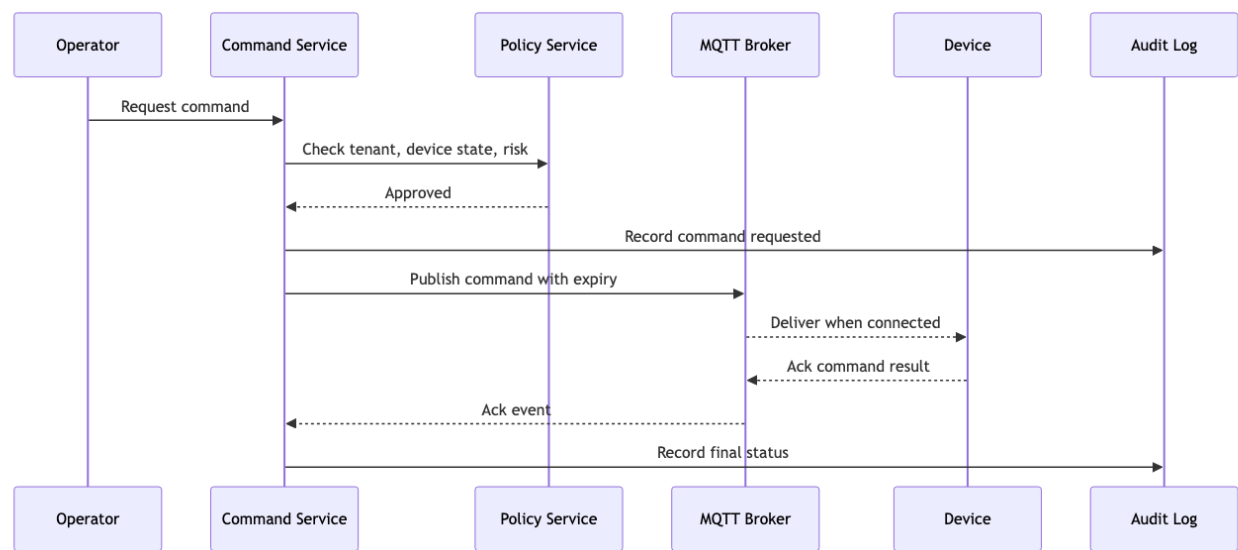
Hình 2: Sơ đồ 61

Pipeline này phải xử lý backpressure. Nếu dashboard chậm, ingestion không được chết. Nếu anomaly detector fail, raw telemetry vẫn cần được lưu. Đây là lý do tách consumer theo event-driven thay vì gọi sync chain dài.

8. Command delivery flow

Command không giống request HTTP thông thường. Device có thể offline 3 giờ. Vì vậy command cần trạng thái rõ ràng:

- Created.
- Authorized.
- Queued.
- Delivered.
- Acked.
- Failed.
- Expired.
- Cancelled.



Hình 3: Sơ đồ 62

9. Device identity and security

IoT security khó vì device nằm ngoài vùng kiểm soát vật lý.

Design principles

- Mỗi device có identity riêng, không dùng shared secret toàn fleet.
- Mutual TLS hoặc certificate-based authentication cho device quan trọng.
- Firmware phải signed.
- Credential rotation phải có kế hoạch trước khi deploy hàng triệu device.
- Device bị compromise phải revoke được.
- Command nguy hiểm cần authorization theo tenant, role, site và device state.

Security boundaries

Boundary	Risk	Mitigation
Device to broker	Fake device gửi telemetry	Mutual auth, certificate pinning
Gateway to cloud	Gateway bị chiếm	Per-gateway identity, scoped permissions
Command API	Operator gửi nhầm command	RBAC, approval, audit
Firmware update	Malicious firmware	Signature verification
Tenant data	Cross-tenant leak	Tenant isolation in topic and storage

10. Data strategy

Telemetry có volume lớn. Nếu lưu tất cả vào database nóng, cost sẽ nổ.

Data type	Store	Retention
Raw telemetry	Data lake/object storage	1-7 năm
Recent telemetry	Time-series DB	7-30 ngày
Aggregates	Warehouse	1-5 năm
Device metadata	Relational DB	Full lifecycle
Command audit	Append-only log	7 năm hoặc theo compliance
Firmware artifacts	Object storage	Theo version policy

Hot path và cold path phải tách nhau. Dashboard cần recent data nhanh. Audit và analytics dài hạn cần data lake rẻ hơn.

11. Failure modes

Failure	Symptom	Mitigation
Network partition	Device offline hàng giờ	Local cache, retry, command expiry
Broker overload	Event lag tăng	Partition by tenant/site, autoscale consumer
Device clock drift	Timestamp sai	Server receive time, correction logic
Duplicate telemetry	Count sai	Idempotency key, dedup window
Bad firmware rollout	Device lỗi hàng loạt	Canary, staged rollout, rollback
Certificate expiry	Device không kết nối được	Rotation workflow, expiry dashboard
Hot store cost spike	Cloud bill tăng	Retention policy, downsampling
Command sent to wrong device	Safety incident	Policy guard, approval, audit

12. ADR examples

ADR 1: Use MQTT broker and event bus for telemetry

Context: Device gửi telemetry liên tục, network không ổn định, consumers cần scale độc lập.

Decision: Device dùng MQTT/HTTP vào ingestion layer. Cloud chuyển telemetry vào event bus để validation, storage, alerting và analytics consume độc lập.

Consequences:

- Tăng resilience và decoupling.
- Cần vận hành broker và event bus.
- Debug flow khó hơn sync API, cần tracing theo message id.

ADR 2: Put safety-critical rules at edge

Context: Một số rule phải chạy ngay cả khi mất cloud connection.

Decision: Edge gateway chạy local rules cho safety-critical scenarios, cloud chỉ cấu hình policy và nhận telemetry.

Consequences:

- Tăng safety khi offline.
- Gateway phức tạp hơn.
- Cần versioning và rollout cho edge rules.

ADR 3: Use tiered storage for telemetry

Context: Telemetry volume lớn, dashboard cần recent data nhanh nhưng audit cần raw data lâu dài.

Decision: Recent data vào time-series DB, raw data vào data lake, aggregates vào warehouse.

Consequences:

- Cost kiểm soát tốt hơn.
- Query model phức tạp hơn.
- Cần data catalog và retention policy rõ.

13. What to document

Một architecture doc IoT tối thiểu nên có:

- Device lifecycle diagram.
- Telemetry C&C view.
- Command sequence diagram.
- Allocation view: device, edge, broker, cloud, data stores.
- Topic naming convention.
- Device identity model.
- Firmware rollout ADR.
- Storage retention table.
- Failure mode table.

14. Self-check

1. Nếu device offline 6 giờ, telemetry và command xử lý thế nào?
2. Command có expiry và audit không?
3. Device identity là per-device hay shared secret?
4. Data nào cần hot store, data nào nên cold store?
5. Nếu anomaly detector fail, ingestion có tiếp tục không?
6. Firmware rollout có canary không?
7. Edge gateway có chạy logic nào không, hay chỉ relay?
8. Làm sao biết topic hoặc tenant nào đang gây backlog?

15. Tóm tắt

- IoT architecture bị drive bởi reliability, scalability, security, observability và safety.
- Event-driven là core vì device communication tự nhiên là async.
- Pipeline phù hợp cho telemetry validation, enrichment, storage và anomaly detection.
- Edge-cloud split là quyết định kiến trúc quan trọng nhất.
- Command delivery cần state machine, không nên coi như HTTP request đơn giản.
- Device identity, firmware signing và credential rotation phải thiết kế từ đầu.
- Storage tiering là bắt buộc nếu telemetry volume lớn.

Bài tiếp: [9.3 Software Architecture for Web3](#).

9.3 Software Architecture for Web3

Tóm tắt một dòng: Web3 architecture là bài toán chia hệ thống thành on-chain và off-chain components, nơi on-chain tối ưu trust và transparency nhưng hy sinh latency, cost, privacy và khả năng sửa lỗi.

1. Domain context

Một công ty muốn xây nền tảng **decentralized carbon credit marketplace**. Doanh nghiệp có thể mua carbon credits, project owner có thể mint credits sau khi được verify, auditor xác nhận chất lượng project, và người dùng có thể xem lịch sử ownership mình bạch.

Yêu cầu business nghe giống marketplace thông thường, nhưng Web3 thêm constraint đặc biệt:

- Một phần state nằm trên blockchain.
- Smart contract gần như immutable sau khi deploy.
- Transaction có gas fee.
- User ký giao dịch bằng wallet.
- Finality không tức thì.
- Public chain làm dữ liệu minh bạch nhưng privacy khó hơn.
- Backend không còn là authority tuyệt đối.

Architecture challenge: quyết định cái gì đưa on-chain, cái gì giữ off-chain.

2. Functional requirements

Marketplace

- List carbon credits.
- Buy/sell/transfer credits.
- Retire credits khi doanh nghiệp claim offset.
- Xem ownership history.

Verification

- Project owner submit project documents.
- Auditor review documents.
- Approved project được mint credits.
- Reject hoặc request changes.

Wallet and identity

- User connect wallet.
- Link wallet với organization account.
- Role-based permissions cho organization admin, trader, auditor.

Indexing and search

- Search credits theo project type, country, vintage, status.
- Show transaction history nhanh.
- Show portfolio dashboard.

Compliance

- Audit trail cho approval và retirement.
- KYC/KYB cho organizations.
- Report export.

3. On-chain vs off-chain decision

Đây là quyết định kiến trúc trung tâm.

Concern	On-chain?	Lý do
Credit token ownership	Có	Cần transparency và trustless transfer
Mint/retire rules	Có	Cần enforce công khai, tránh double spend
Project documents	Không	Dữ liệu lớn, private, thay đổi được
Document hash	Có	Chứng minh document không bị sửa
Search index	Không	Query blockchain trực tiếp chậm và đắt
User profile	Không	Privacy, KYC, mutable data
Audit approval workflow	Hybrid	Decision hash on-chain, workflow off-chain
Pricing/order book	Tuỳ	Nếu cần trustless matching thì on-chain, nếu cần UX tốt thì off-chain

Rule of thumb:

- Put on-chain những gì cần shared trust, settlement, ownership hoặc public verification.
- Keep off-chain những gì cần privacy, high throughput, low latency, rich query hoặc frequent change.

4. Quality Attributes

Priority	QA	Target
Must	Security	Không mất token, không bypass mint/retire rule
Must	Integrity	Ownership và retirement không double count
Must	Auditability	Trace được lifecycle của credit
Must	Correctness	Smart contract invariant được test và verified
Should	Usability	User không bị kẹt vì pending transaction khó hiểu
Should	Performance	Dashboard p95 < 2s dù chain query chậm
Should	Evolvability	Upgrade path rõ cho contract và backend

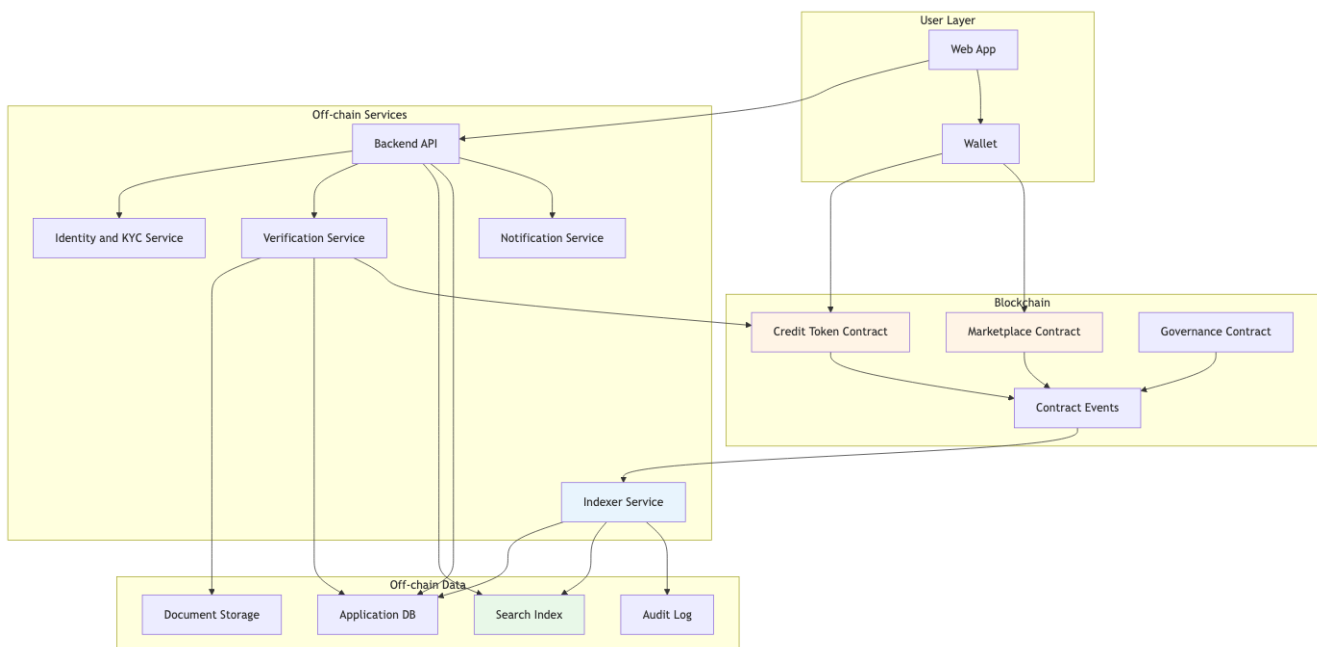
Priority	QA	Target
Should	Cost	Gas cost predictable, batch khi phù hợp
Should	Privacy	KYC và sensitive docs không public

5. Architecture style mix

Concern	Style	Lý do
Smart contracts	Layered domain core on-chain	Contract chứa invariant quan trọng
Blockchain events	Event-Driven	Contract emit events, backend index async
Off-chain backend	Service-based	Marketplace, Identity, Verification, Indexer, Notification
Indexing pipeline	Pipeline	Read event, decode, enrich, persist, project views
Frontend	Layered	Wallet adapter, API client, UI state
Governance modules	Microkernel-like	Add policy module hoặc auditor plugin

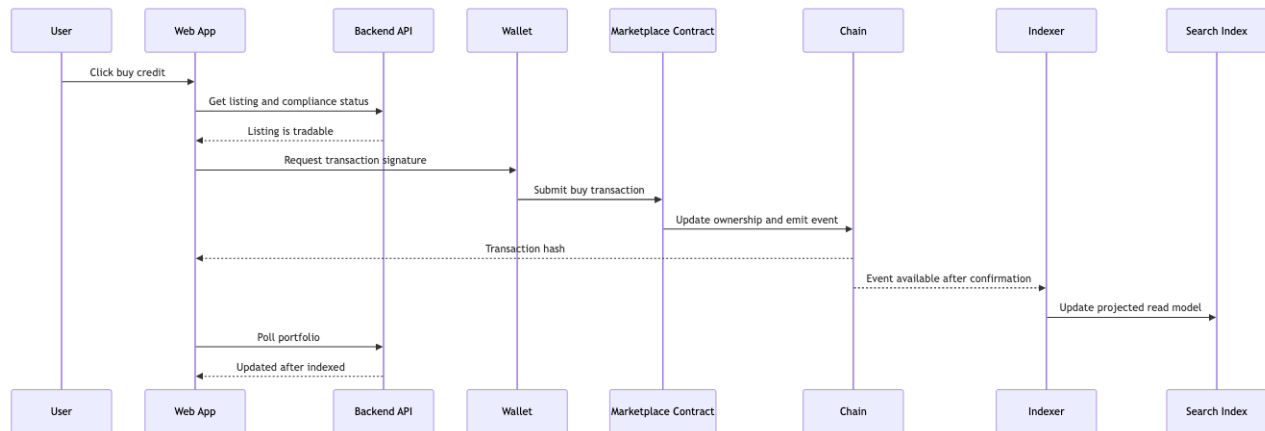
Decision tổng: **Hybrid on-chain/off-chain architecture with event-driven indexer.**

6. High-level architecture



Hình 4: Sơ đồ 63

7. Critical flow: buying a credit



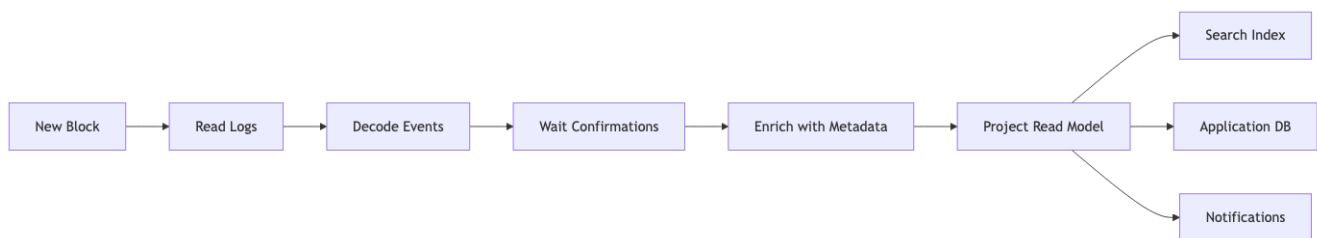
Hình 5: Sơ đồ 64

Điểm UX quan trọng: transaction submitted không có nghĩa là business state đã final. UI cần phân biệt:

- Signed.
- Submitted.
- Pending confirmation.
- Confirmed on-chain.
- Indexed off-chain.
- Failed or reverted.

8. Indexer pipeline

Blockchain không phải database query tốt cho UI. Indexer tạo read model.



Hình 6: Sơ đồ 65

Indexer phải handle:

- Chain reorg.
- Duplicate events.
- Missed blocks.
- Contract version changes.
- Backfill từ block cũ.
- Idempotent projection.

Nếu indexer sai, UI sai dù chain đúng. Vì vậy indexer là component critical, không phải background script phụ.

9. Smart contract as architectural boundary

Smart contract là boundary cứng hơn API thông thường vì:

- Deploy rồi khó sửa.
- Bug có thể mất tiền thật.
- State public.
- Execution cost tính bằng gas.
- External caller không luôn đáng tin.

Contract invariants

Invariant	Ví dụ
No double retirement	Một credit retired thì không transfer được nữa
Mint only approved project	Chỉ role/minter hợp lệ mint được
Conservation	Tổng credits không tự tăng ngoài mint rule
Ownership	Chỉ owner hoặc approved operator transfer được
Pause safety	Có thể pause marketplace khi incident

Những invariant này phải được test mạnh hơn backend logic thông thường: unit test, property-based test, audit, formal verification nếu value lớn.

10. Upgrade strategy

Smart contract immutable không có nghĩa là hệ không evolve. Nhưng upgrade phải được thiết kế.

Strategy	Lợi	Giá phải trả
No upgrade	Trust cao, đơn giản	Bug khó sửa
Proxy upgrade	Có thể sửa logic	Governance risk, complexity
Versioned contracts	Rõ ràng, an toàn hơn proxy	Fragmented liquidity/state
Pause and migrate	Có emergency path	UX phức tạp

Với marketplace có value cao, thường dùng versioned contracts hoặc proxy có governance chặt:

- Timelock.
- Multisig.
- Public proposal.
- Emergency pause.
- Audit trước upgrade.

11. Security model

Web3 security là security architecture, không chỉ smart contract audit.

Layer	Threat	Mitigation
Smart contract	Reentrancy, access control bug	Audit, tests, battle-tested libraries

Layer	Threat	Mitigation
Wallet UX	User ký nhầm transaction	Human-readable signing, simulation
Backend API	Fake compliance status	Backend cannot bypass contract invariant
Indexer	Wrong projected state	Reconciliation against chain
Admin key	Key compromise	Multisig, hardware wallet, timelock
Frontend	DNS or script injection	CSP, integrity, deployment controls
Oracle/auditor	False verification	Multi-party approval, audit trail

Nguyên tắc: backend có thể hỗ trợ UX, nhưng không được là single point làm sai ownership invariant.

12. Privacy and data placement

Public blockchain làm transparency tốt hơn, nhưng privacy khó hơn.

Không nên đưa lên chain:

- KYC documents.
- Legal contracts raw.
- Personal identity.
- Sensitive business volume nếu không cần public.
- Internal approval notes.

Có thể đưa lên chain:

- Hash của document.
- Token ID.
- Project ID public.
- Retirement certificate hash.
- Ownership event.

13. Failure modes

Failure	Symptom	Mitigation
Contract bug	Asset stuck hoặc lost	Audit, pause, limited scope, bug bounty
Chain congestion	Transaction pending lâu	Gas strategy, UX state, retry guidance
Indexer lag	UI stale	Lag dashboard, fallback chain read for critical state
Chain reorg	Event bị revert	Confirmation depth, idempotent projection
Wallet phishing	User mất asset	Signing clarity, education, allowlist
Admin key compromise	Malicious upgrade	Multisig, timelock, separation of duties

Failure	Symptom	Mitigation
Backend outage	UI không search được	Chain state vẫn source of truth, cache read model
Oracle fraud	Mint fake credits	Multi-auditor workflow, stake/slashing nếu phù hợp

14. ADR examples

ADR 1: Store token ownership on-chain, documents off-chain

Context: Ownership cần trustless verification, nhưng project documents lớn và private.

Decision: Token ownership và retirement state nằm on-chain. Documents nằm off-chain, chỉ hash đưa on-chain.

Consequences:

- Ownership minh bạch.
- Privacy tốt hơn.
- Cần document storage bền vững và hash verification.

ADR 2: Build an indexer instead of querying chain directly from UI

Context: UI cần search/filter nhanh, blockchain query chậm và không phù hợp với dashboard.

Decision: Indexer consume contract events, project read model vào search index và application DB.

Consequences:

- UX nhanh hơn.
- Có eventual consistency giữa chain và UI.
- Cần handle reorg và reconciliation.

ADR 3: Use multisig and timelock for contract upgrades

Context: Contract có thể cần upgrade, nhưng admin key là risk lớn.

Decision: Upgrade qua multisig + timelock + public announcement.

Consequences:

- Giảm risk key compromise.
- Upgrade chậm hơn.
- Emergency response cần pause mechanism riêng.

15. What to document

Một architecture doc Web3 tối thiểu nên có:

- On-chain/off-chain responsibility table.
- Contract invariant list.
- Contract upgrade policy.
- Event indexing flow.
- Finality and reorg policy.

- Wallet transaction UX states.
- Admin key governance.
- Threat model.
- Data privacy placement.
- Incident response playbook.

16. Self-check

1. State nào thật sự cần on-chain?
2. Nếu smart contract bug, rollback path là gì?
3. UI lấy data từ chain trực tiếp hay indexer?
4. Indexer handle reorg thế nào?
5. User biết transaction đang pending hay confirmed không?
6. Admin key được bảo vệ bằng gì?
7. Dữ liệu private có bị đưa lên public chain không?
8. Backend có thể gian lận ownership không, hay contract chặn được?

17. Tóm tắt

- Web3 không thay backend, mà thêm một trust layer on-chain.
- On-chain tối ưu trust, transparency và settlement, nhưng hy sinh latency, cost, privacy và evolvability.
- Off-chain services vẫn cần cho identity, search, indexing, documents, notifications và UX.
- Indexer là event-driven projection từ chain sang read model.
- Smart contract là architectural boundary cứng, cần invariant, audit và upgrade strategy.
- Governance và key management là phần của architecture, không phải chi tiết vận hành nhỏ.

Bài tiếp: [9.4 Software Architecture for MLOps](#).

9.4 Software Architecture for MLOps

Tóm tắt một dòng: MLOps architecture là cách tổ chức toàn bộ vòng đời ML thành một production system có data contracts, reproducibility, deployment, monitoring, rollback và governance, không chỉ là train model trong notebook.

1. Domain context

Một công ty thương mại điện tử có 15 Data Scientists, 8 ML Engineers và 6 Data Engineers. Công ty đang vận hành nhiều use cases:

- Search ranking.
- Product recommendation.
- Fraud detection.
- Demand forecasting.
- Churn prediction.
- Dynamic pricing.
- Customer support automation.

Ban đầu mỗi team tự tạo pipeline riêng. Sau một thời gian, các vấn đề xuất hiện:

- Notebook khó reproduce.
- Feature logic duplicate.
- Model deploy thủ công.
- Không có rollback chuẩn.
- Drift không được detect sớm.
- Training data và serving data khác nhau.
- Không truy được prediction dùng model version nào.
- Compliance hỏi lineage nhưng team không trả lời đủ.

MLOps platform ra đời để biến ML từ craft cá nhân thành engineering system.

2. Functional requirements

Data and feature

- Ingest data từ warehouse, event stream, object storage.
- Validate data schema và quality.
- Quản lý feature definitions.
- Build training dataset point-in-time correct.
- Materialize online features cho low-latency serving.

Experiment and training

- Track experiment parameters, metrics, artifacts.
- Run training pipeline reproducible.
- Support CPU/GPU jobs.
- Store model artifact và metadata.
- Compare models trước khi promote.

Deployment and serving

- Deploy model as batch job, online API hoặc streaming consumer.

- Canary, shadow và A/B test.
- Rollback nhanh.
- Serve multiple model versions.
- Log prediction với feature versions.

Monitoring and feedback

- Monitor model latency, error, throughput.
- Monitor feature drift, prediction drift, data quality.
- Collect delayed labels.
- Trigger retraining hoặc review.
- Dashboard cho DS, ML Engineer và business owner.

Governance

- Approval workflow cho model production.
- Model card, risk level, owner.
- PII policy và access control.
- Audit lineage end-to-end.

3. Quality Attributes

Priority	QA	Target
Must	Reproducibility	Rebuild được model từ code, data, feature, config version
Must	Observability	Drift, latency, error, prediction distribution có dashboard
Must	Deployability	Promote/rollback model trong < 15 phút
Must	Auditability	100% production predictions có model and feature lineage
Must	Data quality	Schema/null/range checks trước training và serving
Should	Scalability	Training jobs và serving scale độc lập
Should	Cost control	GPU usage tracked per team/model
Should	Security	PII policy enforced ở feature và logs
Should	Extensibility	Thêm framework/model type không rewrite platform

4. Architecture style mix

Concern	Style	Lý do
Training pipeline	Pipeline	Data prep, feature, train, evaluate, register
Event feedback	Event-Driven	Prediction logs, labels, drift alerts, retrain triggers
Platform core	Service-based	Feature, registry, training, serving, monitoring services
Model serving	Microkernel-like	Runtime adapters for sklearn, PyTorch, XGBoost, LLM
UI/API	Layered	Portal, API, business logic, persistence
Multi-team platform	Modular monolith or service-based first	Avoid premature microservices

Decision tổng: **Service-based MLOps platform with pipeline orchestration and event-driven feedback loops.**

5. High-level architecture

6. Core platform boundaries

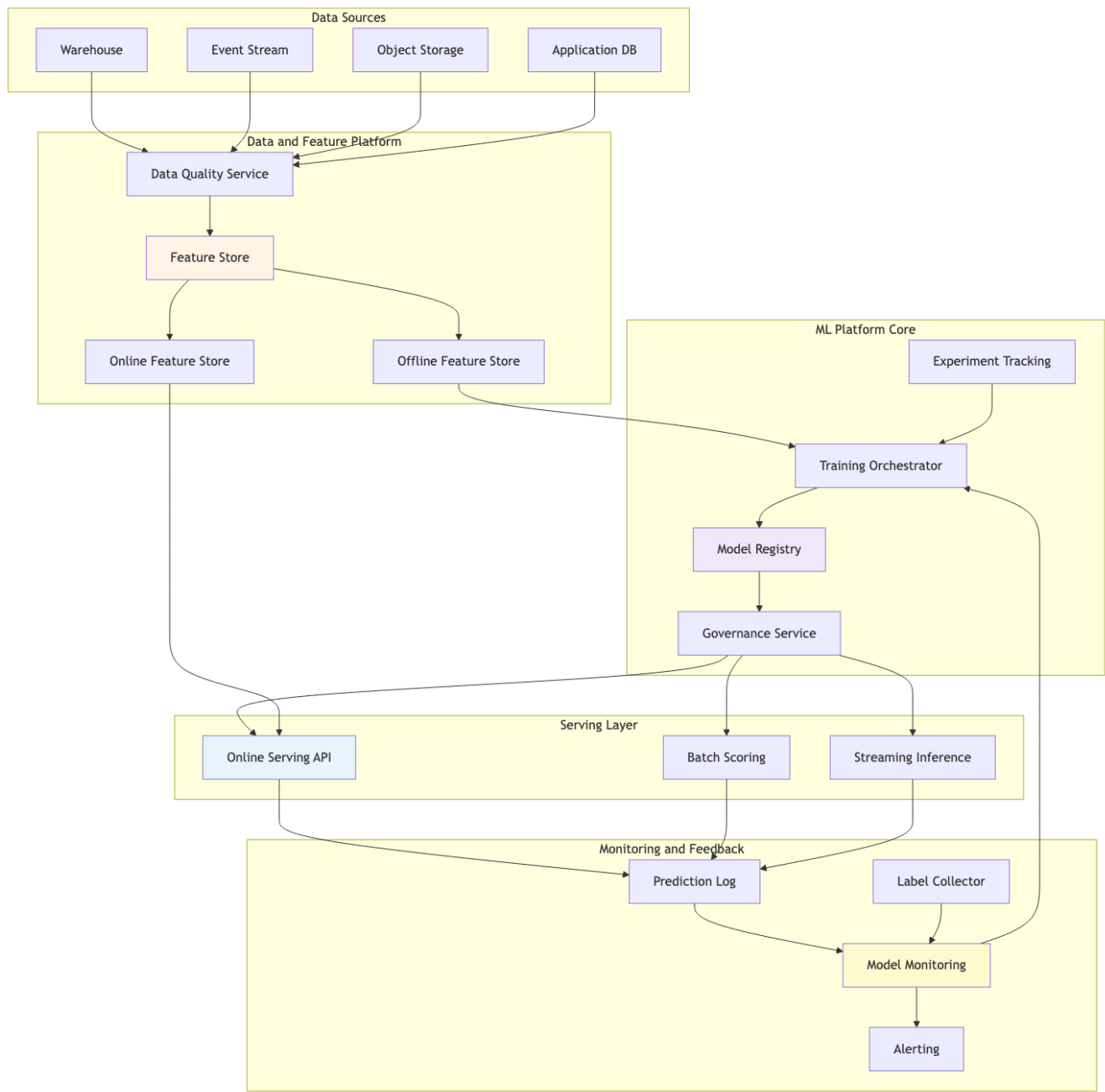
Service	Responsibility	Owns
Data Quality Service	Schema, null, range, anomaly checks	Data quality rules and results
Feature Store	Feature definitions, versions, materialization metadata	Feature metadata
Experiment Tracking	Runs, params, metrics, artifacts	Experiment records
Training Orchestrator	Pipeline execution and compute scheduling	Training jobs
Model Registry	Model artifacts, versions, lifecycle states	Model metadata and artifacts
Governance Service	Approval, risk level, policy gates	Promotion workflow
Serving API	Online inference and model runtime	Runtime config
Monitoring Service	Drift, latency, error, business feedback	Monitoring metrics

Boundary tốt giúp DS không cần tự build deployment path, và platform team không cần hiểu chi tiết từng model để enforce policy.

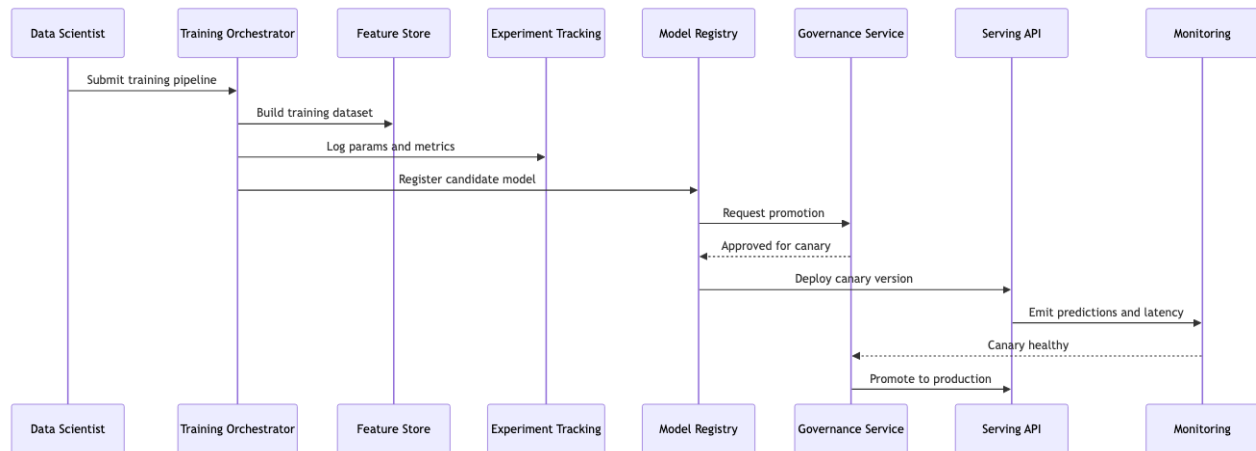
7. Training-to-serving lifecycle

Pipeline này biến model deployment thành workflow có state rõ ràng, không phải copy artifact thủ công.

8. Deployment modes



Hình 7: Sơ đồ 66



Hình 8: Sơ đồ 67

Mode	Use case	Architecture implication
Batch scoring	Churn, demand forecast	Schedule, warehouse output, lower latency pressure
Online serving	Recommendation, pricing	Low latency, online feature store, autoscaling
Streaming inference	Fraud, realtime personalization	Event bus, stateful stream processing, exactly-once concerns
Edge inference	Mobile/IoT	Model compression, offline update, device allocation view

Một MLOps platform tốt không ép mọi model vào cùng một serving mode. Nó cung cấp common lifecycle nhưng cho phép deployment topology khác nhau.

9. Model registry lifecycle

Registry không chỉ là nơi lưu file model. Nó là state machine quản lý model lifecycle.

10. Monitoring design

Production ML monitoring có nhiều lớp.

Layer	Metrics
Data quality	schema changes, null rate, out-of-range, freshness
Feature Serving	drift, skew, online/offline mismatch, missing value latency, throughput, error rate, timeout, resource usage
Prediction Label feedback	distribution shift, confidence shift, class balance delayed accuracy, precision, recall, business metric
Business	revenue, fraud loss, churn reduction, user engagement

Risk tier	Example	Required gate
Critical	Safety system	Formal validation, staged rollout, manual override

Governance là kiến trúc vì nó ảnh hưởng deployment flow, metadata, audit log và ownership.

12. Failure modes

Failure	Symptom	Mitigation
Data schema change	Training or serving fail	Data contracts, schema registry, quality gates
Training-serving skew	Offline metric tốt, online metric tệ	Shared feature definitions, feature version check
Bad model deployed	Business metric giảm	Canary, shadow, rollback
Drift	Prediction distribution thay đổi	Drift monitoring, retraining trigger
Label delay	Không biết model tệ ngay	Proxy metrics, delayed evaluation
GPU cost spike	Cloud bill tăng	Quota, chargeback, job scheduling
PII leakage	Compliance incident	Feature policy, log redaction, access control
Pipeline not reproducible	Không rebuild được model	Code/data/config/artifact versioning

13. ADR examples

ADR 1: Use model registry as lifecycle authority

Context: Models are deployed manually and production status is unclear.

Decision: Model Registry is the source of truth for model artifact, version, stage and promotion state.

Consequences:

- Deployment becomes auditable.
- Serving must integrate with registry.
- Teams must follow promotion workflow.

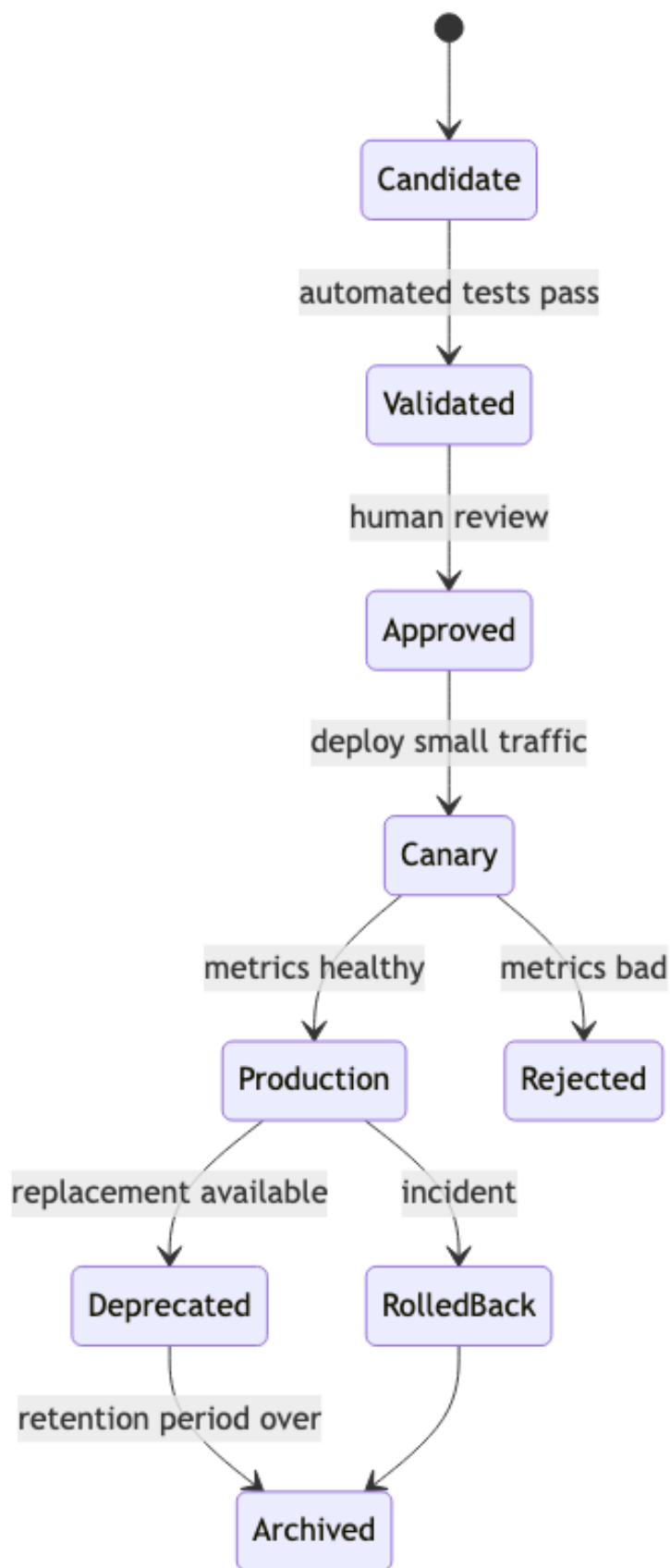
ADR 2: Support multiple serving modes

Context: Fraud needs streaming inference, churn needs batch, recommendation needs online.

Decision: Platform provides batch, online and streaming serving adapters under one registry/governance lifecycle.

Consequences:

- Better fit for diverse ML use cases.
- Platform complexity increases.



- Monitoring must normalize metrics across modes.

ADR 3: Enforce feature version compatibility at serving load time

Context: Training-serving skew caused incidents.

Decision: Serving API checks model's required feature versions against online feature store metadata before loading model.

Consequences:

- Prevents many silent correctness bugs.
- Deployment can fail earlier if feature materialization lags.
- Requires feature registry maturity.

14. Documentation checklist

Một architecture doc MLOps tối thiểu nên có:

- Model lifecycle state diagram.
- Training pipeline C&C view.
- Serving flow sequence diagram.
- Feature store boundary.
- Monitoring metrics table.
- Governance gates.
- Rollback playbook.
- Allocation view for CPU/GPU and online serving.
- Data lineage model.
- Cost ownership model.

15. Self-check

1. Model version nào đang production, ai approved?
2. Prediction log có model version và feature versions không?
3. Nếu data schema đổi, pipeline fail ở đâu?
4. Rollback model mất bao lâu?
5. Drift được detect bằng metric nào?
6. Batch, online và streaming inference có cùng lifecycle không?
7. Training dataset có point-in-time correctness không?
8. GPU cost có owner không?
9. PII có bị log trong prediction trace không?
10. DS có thể reproduce model sau 6 tháng không?

16. Tóm tắt

- MLOps là Software Architecture cho vòng đời ML.
- Model registry, feature store, training orchestrator, serving và monitoring là platform boundaries.
- Pipeline Architecture phù hợp training và data preparation.
- Event-Driven Architecture phù hợp prediction logs, labels, drift alerts và retraining.
- Service-based core phù hợp team platform vừa và lớn.
- Reproducibility, deployability, auditability và observability quan trọng không kém accuracy.
- Governance là một phần của architecture, không phải thủ tục giấy tờ.

Bài tiếp: [9.5 Software Architecture for Digital Twin](#).

9.5 Software Architecture for Digital Twin

Tóm tắt một dòng: Digital Twin architecture là bài toán giữ một mô hình số đủ đồng bộ với physical system để quan sát, mô phỏng, dự đoán và đôi khi điều khiển thế giới thật một cách an toàn.

1. Domain context

Một tập đoàn sản xuất muốn xây **Digital Twin Platform** cho 20 nhà máy. Mục tiêu là tạo bản sao số của dây chuyền sản xuất, máy móc và luồng vận hành để:

- Theo dõi trạng thái real-time.
- Mô phỏng bottleneck.
- Dự đoán hỏng hóc.
- Tối ưu lịch bảo trì.
- Thử kịch bản thay đổi production plan.
- Gửi recommendation hoặc command về hệ thống điều khiển.

Digital Twin không chỉ là dashboard IoT đẹp hơn. Nó là hệ thống có model của physical world: asset hierarchy, state, behavior, constraints và simulation.

2. Physical twin vs digital twin

Physical world	Digital representation
Machine	Asset entity
Sensor reading	Telemetry event
Production line	Topology graph
Machine state	Twin state
Maintenance action	Event and workflow
Control command	Actuation request
Physics/process rule	Simulation model
Failure prediction	ML model output

Điểm khó: digital twin luôn trễ hoặc không đầy đủ so với physical world. Architecture phải quyết định độ trễ chấp nhận được và cách xử lý uncertainty.

3. Functional requirements

Twin modeling

- Quản lý asset hierarchy: factory, line, machine, component.
- Quản lý relationship giữa assets.
- Quản lý twin type: pump, motor, conveyor, robot arm.
- Gắn schema telemetry và state variables cho mỗi type.

Real-time sync

- Ingest telemetry từ sensors, PLC, SCADA, MES.
- Update twin state gần real-time.
- Detect stale state.
- Store time-series history.

Analytics and simulation

- Run anomaly detection.
- Predict failure risk.
- Simulate production throughput.
- Compare what-if scenarios.
- Recommend maintenance action.

Visualization

- 2D/3D view factory.
- Timeline playback.
- Alert dashboard.
- Drill-down từ factory tới component.

Closed-loop action

- Send recommendation to operator.
- Optionally send command to control system.
- Require approval for risky action.
- Audit all decisions and commands.

4. Quality Attributes

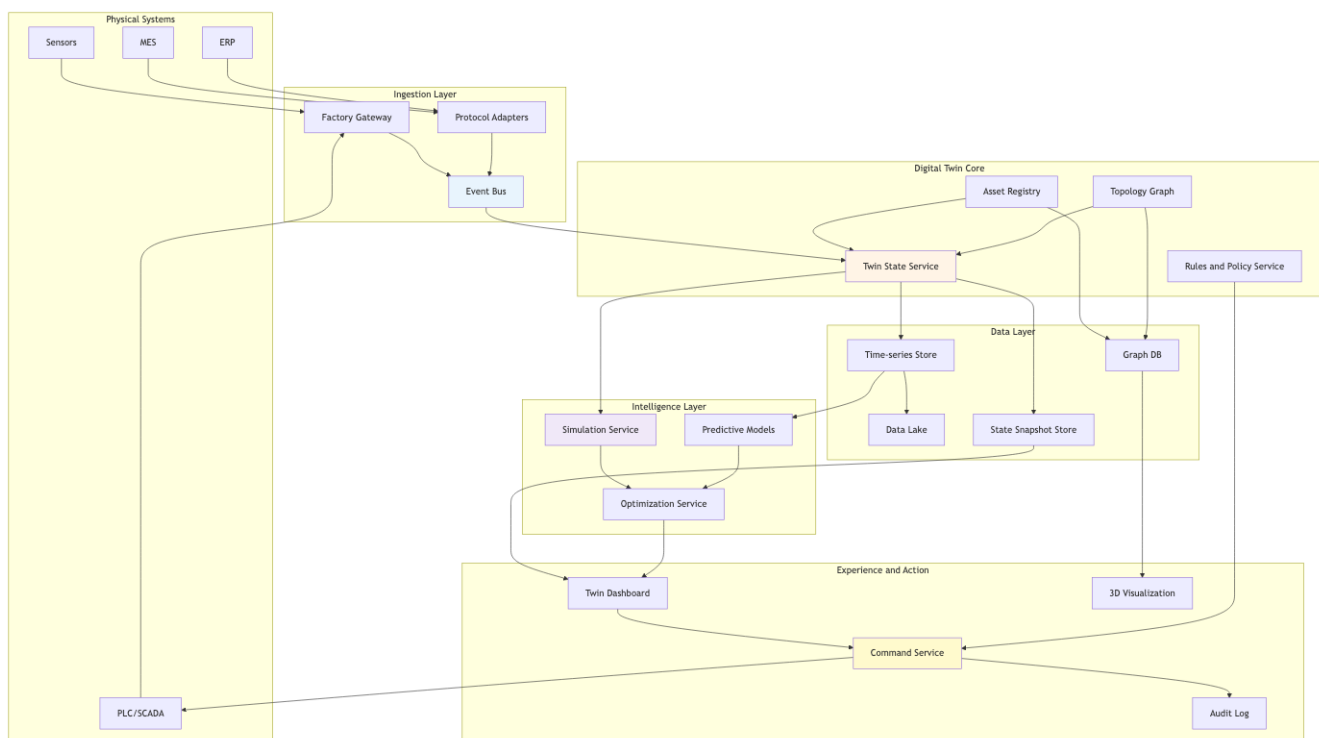
Priority	QA	Target
Must	State freshness	Critical assets updated < 5s
Must	Reliability	Telemetry loss visible and recoverable
Must	Safety	No unsafe command without policy and approval
Must	Traceability	Every twin state derived from source events
Must	Scalability	20 factories, 1M assets, 100k events/s peak
Should	Simulation performance	What-if scenario result < 2 minutes for planning use case
Should	Evolvability	Add new asset type without rewriting platform
Should	Observability	Lag, stale state, model error, command status visible
Should	Interoperability	Integrate SCADA, MES, ERP, IoT protocols

5. Architecture style mix

Concern	Style	Lý do
Telemetry sync	Event-Driven	Physical events update twin asynchronously
State update pipeline	Pipeline	Validate, map, enrich, update, alert
Twin model platform	Service-based	Asset, topology, state, simulation, command services
Asset type extensions	Microkernel	Plugin for new machine types or simulation models
Visualization app	Layered	UI, API, query model
Simulation workloads	Distributed batch/compute	Heavy scenario runs need isolated compute

Decision tổng: **Event-driven digital thread with service-based twin core and simulation plugins.**

6. High-level architecture



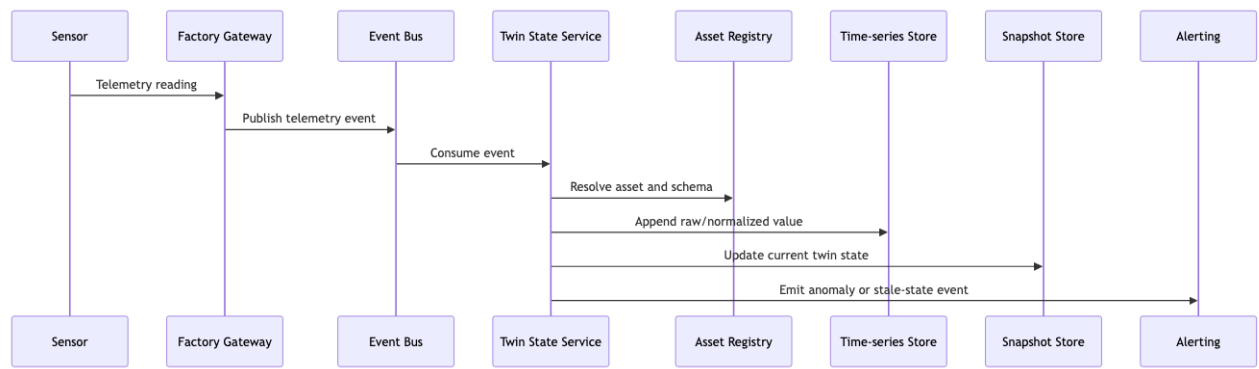
Hình 10: Sơ đồ 69

7. Twin state update flow

State update phải idempotent. Telemetry có thể duplicate hoặc arrive out of order. Twin state service cần version, event timestamp và source timestamp để xử lý.

8. Digital thread

Digital Twin tốt cần **digital thread**: khả năng nối từ event thô tới state, prediction, recommendation và action.



Hình 11: Sơ đồ 70



Hình 12: Sơ đồ 71

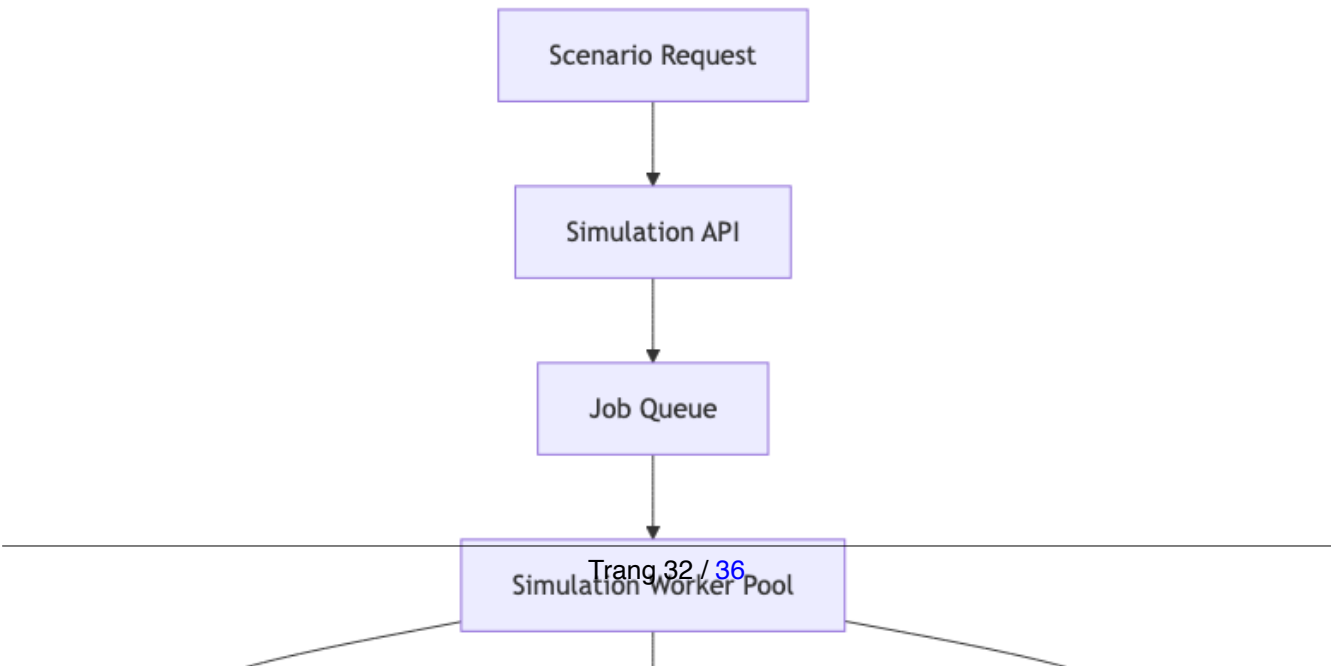
Nếu thiếu thread này, dashboard có thể đẹp nhưng không audit được quyết định. Khi recommendation sai, bạn không biết input nào, model nào, simulation nào dẫn tới action đó.

9. Simulation architecture

Simulation có nhiều loại:

Simulation type	Latency need	Example
Real-time state estimation	Seconds	Estimate hidden machine state
What-if planning	Minutes	Change production schedule
Physics-based simulation	Minutes to hours	Thermal or fluid model
Discrete event simulation	Minutes	Factory throughput bottleneck
ML surrogate model	Milliseconds to seconds	Fast approximation for optimization

Không nên ép mọi simulation vào real-time path. Critical dashboard dùng current state và lightweight model. Planning simulation có thể chạy async job.



10. Open-loop vs closed-loop control

Digital Twin có thể chỉ observe, recommend hoặc control.

Mode	Description	Risk
Observe	Chỉ hiển thị state	Thấp
Alert	Cảnh báo operator	Thấp-vừa
Recommend	Đề xuất hành động	Vừa
Human-approved command	Operator approve rồi gửi command	Cao
Autonomous control	System tự điều khiển	Rất cao

Architecture nên tiến hoá theo maturity. Đừng nhảy thẳng tới autonomous control nếu observability và audit còn yếu.

11. Safety and command policy

Command về physical system cần guardrail.

Guardrail	Example
State precondition	Chỉ restart máy nếu không ở trạng thái active production
Role approval	Maintenance lead phải approve
Time window	Không update trong giờ cao điểm
Rate limit	Không gửi quá N commands/site/hour
Simulation check	Command phải pass what-if safety rule
Manual override	Operator có thể stop command
Audit	Ghi lại recommendation, approver, command, outcome

Command Service không nên nói chuyện trực tiếp với PLC nếu chưa qua Policy Service.

12. Data modeling

Digital Twin cần cả graph, time-series và document/object storage.

Data	Store	Reason
Asset hierarchy	Graph DB hoặc relational with hierarchy	Query relationship
Current state	Snapshot store	Fast dashboard
Telemetry history	Time-series DB	Time-window query
Raw events	Data lake	Audit, replay, training
Simulation model	Model registry/object storage	Versioned model artifacts
Scenario result	Object/document store	Large result payload
Command audit	Append-only log	Compliance and safety

Không có một database duy nhất fit mọi loại query. Đây là polyglot persistence có lý do, không phải chạy theo trend.

13. Consistency and freshness

Twin state luôn là approximation. Cần hiển thị freshness rõ ràng.

State	Meaning	UI behavior
Fresh	Updated within SLA	Normal
Stale	No update beyond SLA	Yellow warning
Unknown	No reliable data	Grey or unavailable
Conflicting	Sources disagree	Red warning and require investigation
Simulated	Not observed, estimated	Label clearly

Nếu UI hiển thị state stale như state fresh, người vận hành có thể quyết định sai.

14. Failure modes

Failure	Symptom	Mitigation
Telemetry lag	Twin state stale	Lag metric, stale status, alert
Out-of-order events	State đi ngược	Event time handling, versioning
Wrong asset mapping	Sensor update nhầm machine	Asset registry validation, commissioning workflow
Simulation model wrong	Recommendation sai	Model validation, confidence, human review
Command unsafe	Physical incident	Policy gates, approval, manual override
Graph inconsistency	Dashboard topology sai	Graph validation, change workflow
Data lake missing events	Cannot replay	Ingestion audit, dead letter queue
Visualization hides uncertainty	Operator overtrust	Freshness and confidence display

15. ADR examples

ADR 1: Use event-driven state synchronization

Context: Physical systems produce continuous telemetry and integration sources vary by factory.

Decision: All telemetry and operational changes enter through event bus and update twin state asynchronously.

Consequences:

- Decouples ingestion from consumers.
- Supports replay and new analytics consumers.
- Twin state is eventually consistent, so freshness must be visible.

ADR 2: Use graph model for asset topology

Context: Factory assets have nested and network relationships: line, machine, component, sensor, dependency.

Decision: Store topology in graph-oriented model, expose query API for dashboards and simulation.

Consequences:

- Relationship queries easier.
- Need governance for topology changes.
- Team must learn graph query and validation.

ADR 3: Require human approval for high-risk commands

Context: Optimization service can recommend actions affecting physical production.

Decision: High-risk commands require policy check and human approval before execution.

Consequences:

- Safety improves.
- Automation benefit is lower initially.
- Audit workflow becomes mandatory.

16. What to document

Một architecture doc Digital Twin tối thiểu nên có:

- Asset model and topology diagram.
- Telemetry ingestion C&C view.
- Twin state update sequence.
- Simulation job architecture.
- Command and approval flow.
- Freshness and consistency semantics.
- Data store mapping.
- Safety policy.
- Replay and audit strategy.
- Allocation view across factory edge and cloud.

17. Self-check

1. Twin state fresh, stale, unknown được phân biệt thế nào?
2. Sensor event map vào asset bằng rule nào?
3. Có replay được raw events để rebuild twin state không?
4. Simulation chạy sync hay async?
5. Command nào cần human approval?
6. Nếu model prediction sai, truy được input và model version không?
7. Topology graph thay đổi qua workflow nào?
8. Edge gateway có thể tiếp tục hoạt động khi mất cloud không?
9. Dashboard có hiển thị uncertainty không?
10. Digital Twin chỉ observe, recommend hay control?

18. Tóm tắt

- Digital Twin là sự kết hợp của IoT, event streaming, state management, simulation, analytics và command governance.
- Twin state luôn có độ trễ và uncertainty, nên freshness phải là first-class concept.
- Event-driven sync giúp decouple ingestion và support replay.
- Graph model phù hợp asset topology.
- Simulation nên tách real-time path và planning path.
- Closed-loop control cần safety policy, human approval và audit.
- Digital thread giúp truy từ telemetry tới state, prediction, recommendation, command và outcome.

Kết thúc cụm seminar. Hãy quay lại [Bản đồ phụ thuộc](#) để nối các seminar này với foundation concepts.